# Dynamic AI Knowledge Assistant using Automated Wikipedia Knowledge Base and Advanced RAG

| | |
|---|---|
| **Skills take away From This Project** | • Python Programming<br>• REST API Integration<br>• Web Data Collection<br>• Natural Language Processing (NLP)<br>• Text Cleaning and Preprocessing<br>• Metadata Extraction<br>• Semantic Chunking<br>• Embedding Generation<br>• Vector Databases (FAISS/ChromaDB)<br>• Hybrid Retrieval<br>• Retrieval-Augmented Generation (RAG)<br>• Prompt Engineering<br>• Retrieval Optimization<br>• Reranking Techniques<br>• Context Compression<br>• Conversation Memory<br>• Explainable AI using Source Citations |

| | |
|---|---|
| | • Streamlit Dashboard Development |
| **Domain** | Artificial Intelligence \| Natural Language Processing \| Information Retrieval \| Generative AI \| Knowledge Management |

**Problem Statement:**

Organizations often need AI assistants that can quickly build a knowledge base from publicly available information without requiring manual document collection.

The objective of this project is to develop a **Dynamic AI Knowledge Assistant** that automatically creates its own knowledge base from Wikipedia based on user-provided keywords.

The application should allow users to enter one or more keywords (for example, *Machine Learning*, *Deep Learning*, *Natural Language Processing*, *Large Language Models*), automatically retrieve the most relevant Wikipedia articles, build a searchable vector database, and answer questions using an advanced Retrieval-Augmented Generation (RAG) pipeline.

The assistant should provide accurate responses along with the source articles used to generate the answer.

## Business Use Cases:

### AI Research Assistant

Build an instant AI knowledge base for any technical topic.

---

### Student Learning Assistant

Generate study material automatically for selected subjects.

---

### Enterprise Knowledge Assistant

Create a temporary knowledge base for company projects.

---

### Technical Documentation Assistant

Build searchable documentation from online resources.

---

### Educational Chatbot

Generate subject-specific chatbots without manually collecting documents

GUVI | HCL
Skill Up. Level Up

# Dataset

Unlike traditional RAG systems, no dataset is provided.

The application automatically generates its own dataset using Wikipedia.

**Input**

User enters one or more keywords. Example Machine Learning, Deep Learning, Artificial Intelligence, Large Language Models, Transformers, Natural Language Processing

---

**Data Collection**

The system should:

- Search Wikipedia
- Retrieve relevant articles
- Remove duplicates
- Download article content
- Save articles locally
- Generate metadata

---

**Supported Data Source**

Wikipedia (English)

---

# Project Workflow

---

# Phase 1 – Knowledge Base Creation

User enters⟶ Machine Learning, Deep Learning, Computer Vision

↓

Search Wikipedia

↓

Retrieve Top-N Relevant Articles

↓

Download Articles

↓

Store as Text Files

↓

Generate Metadata

---

Expected Output

knowledge_base/

- Machine_Learning.txt
- Deep_Learning.txt
- Computer_Vision.txt
- CNN.txt
- Transformer.txt
- metadata.csv

---

# Phase 2 – Data Cleaning

Perform

- HTML Removal
- Unicode Normalization
- Duplicate Removal
- Empty Page Removal
- Special Character Cleaning
- Stopword Analysis (Optional)

---

# Phase 3 – Metadata Generation

Generate

- Article Name
- URL
- Category
- Word Count
- Character Count
- Download Date

Save

metadata.csv

---

# Phase 4 – Text Chunking

Students should compare

- Fixed Chunking
- Recursive Chunking
- **Semantic Chunking**

Evaluate

- Number of Chunks
- Retrieval Accuracy
- Processing Time

---

# Phase 5 – Embedding Generation

Create embeddings of documents

---

# Phase 6 – Vector Database

Store embeddings using

- FAISS or
- ChromaDB

---

# Phase 7 – Retrieval Pipeline

Implement

**Basic Similarity Search**

---

# Phase 8 – Prompt Engineering

Design prompts for

- Question Answering
- Summarization
- Concept Explanation
- Comparison
- Example Generation

---

# Phase 9 – Response Generation

Generate responses using

- Llama 3
- Gemma
- Mistral
- GPT (Optional)

The chatbot should display

- Final Answer
- Retrieved Context
- Source Articles
- Confidence Score

GUVI | HCL
Skill Up. Level Up

---

# Phase 10 – Streamlit Dashboard

The application should contain the following pages.

## 1. Dataset Builder

- Enter Keywords
- Download Articles
- View Download Progress
- Display Metadata

---

## 2. Knowledge Base

Display

- Total Articles
- Word Count
- Chunk Count
- Categories
- Downloaded Files

---

## 3. Chat Assistant

Features

- Ask Questions
- Streaming Responses

---

## 4. Analytics Dashboard

Display

- Number of Articles
- Chunk Distribution
- Embedding Statistics
- Retrieval Time
- Response Time
- Vector Database Size

---

---

# Expected Results

The completed solution should

- Automatically build a knowledge base from Wikipedia
- Download relevant articles based on user keywords
- Create a searchable vector database
- Retrieve relevant information efficiently
- Generate grounded responses using RAG
- Provide source citations for every answer
- Support multi-turn conversations
- Allow users to rebuild the knowledge base for different domains without changing the application

---

### System Performance

- Download Time
- Chunking Time
- Embedding Time
- Retrieval Latency
- Response Time

GUVI | HCL
Skill Up. Level Up

---

## Technical Tags

Python • REST API • Wikipedia API • Natural Language Processing • Retrieval-Augmented Generation • LangChain • LlamaIndex • Sentence Transformers • ChromaDB • FAISS • Hybrid Search • Semantic Search • Prompt Engineering • Explainable AI • Streamlit • AWS

---

## Project Deliverables

- Wikipedia article downloader module
- Metadata generation module
- Text preprocessing pipeline
- Chunking strategy comparison notebook

- Embedding generation notebook
- Vector database implementation
- Advanced RAG pipeline
- Prompt engineering experiments
- Multi-turn conversational chatbot
- Streamlit application
- RAG evaluation report
- Cloud deployment
- GitHub repository with modular code
- Comprehensive README
- Final project report
- Project presentation

---

# Project Guidelines

- Accept a minimum of **5 user-defined keywords** to generate the knowledge base.
- Automatically retrieve and store the most relevant Wikipedia articles while avoiding duplicates.
- Build a reusable pipeline for document downloading, preprocessing, chunking, embedding generation, indexing, and retrieval.
- Compare at least **two chunking strategies**, **two embedding models**, and **two vector databases**.
- Include source citations with every generated answer.
- Evaluate retrieval and generation quality using standard RAG metrics.
- Build a modular Streamlit application with separate pages for dataset generation, knowledge base management, chat, analytics, and evaluation.
- Write clean, production-ready code with proper documentation and version control.

**Bonus Features (Optional)**

- Crawl related Wikipedia links to expand the knowledge base automatically.
- Allow users to choose the maximum number of articles per keyword.
- Cache downloaded articles to avoid repeated downloads.
- Generate automatic summaries for each downloaded article.
- Support multilingual Wikipedia.

- Visualize relationships between downloaded topics as a knowledge graph.
- Schedule automatic updates to refresh the knowledge base.

This project is unique because the **dataset is generated dynamically at runtime**, making it reusable for **any technical domain** (AI, Healthcare, Finance, Law, Education, etc.) simply by changing the input keywords. It showcases the complete lifecycle of an enterprise-grade RAG system—from knowledge acquisition and indexing to retrieval, answer generation, evaluation, and deployment—making it a standout portfolio project for Generative AI and LLM engineering roles.

GUVI | HCL
Skill Up. Level Up

**PROJECT DOUBT CLARIFICATION SESSION ( PROJECT AND CLASS DOUBTS)**

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session

aims to enhance comprehension and provide guidance to overcome challenges effectively.

**Note:** Book the slot at least before 12:00 Pm on the same day

**Timing:** Saturday (5:00PM to 7:00PM)

**Booking link :**https://forms.gle/NtkQ4UV9cBV7Ac3C8

**LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)**

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive

real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.

**Note:** This form will Open on Saturday and Sunday Only on Every Week

**Timing:** Monday-Saturday (11:30PM to 12:30PM)

**Booking link :** https://forms.gle/1m2Gsro41fLtZurRA